

Hirukshanan

Application, API and data

Status: Proposed work package, not a Jira assignment, client approval, or evidence of completion. Match accounts and agree the task before starting.

Shared goal and how to use this guide

The intended Sprint 1 goal is: sign in, add a supported competitor listing, request collection, and see a trustworthy observation in your own workspace. Development is planned for 28 September–11 October 2026; evaluation starts 12 October (12–18 October window). Verify course updates. No individual completion date or available hours are assumed.

First connect a page to one backend response labelled **synthetic**. This is a short learning/integration checkpoint, not the full sprint or live Amazon acceptance. Accounts, persistence, jobs and permitted source collection follow only when their dependencies are ready.

Read `docs/manual.md` sections 2–5, then the sections named below. Follow one step at a time; the later steps are a route toward the goal, not an instruction to implement the entire guide unattended. Jira owns actual status and assignments; this PDF is a dated handout.

Your deliverable and boundaries

Build the server behaviour that the UI can rely on, then account-safe storage and product/listing operations. First reviewer: Ilmam; Shewon reviews observation semantics and Dilsan reviews payload usability. Relevant scope: PB-03-05/12, FR-01-04/06/11, NFR-01/05/07/08. Read manual sections 6–8 and 10.

Do not implement an entire backend before the frontend connects. Do not copy old custom auth or SQLite code as an approved architecture. Database/auth choices remain proposed until the appropriate trial and team decision.

First working slice: one HTTP response

1. Explain a request and response in your own words. Agree the manual section 7 sample with Dilsan and Shewon, including nullable price and the synthetic label.
2. With Ilmam's setup, create the proposed demo endpoint returning this sample. It must contain no private retailer records and perform no remote collection.
3. Add a small automated check of the response status and agreed fields/types. Run it and explain both what it proves and what it does not prove.
4. Connect with Dilsan's screen early. Resolve field-name or error-handling differences together; update the shared example once rather than maintaining competing definitions.

Checkpoint evidence: A direct request and the browser receive the agreed synthetic response; your test checks that contract; another member can run it. No claim of database, authentication, or live source support.

Next slices toward the full sprint goal

1. **Persistence trial and model.** Agree PostgreSQL setup with Ilmam. Model a workspace, user/membership, tracked product, monitor, attempt and observation as needed. Explain relationships and ownership before adding tables. Add reviewed migrations; verify a clean database can be created. Store money without binary floating-point calculations.
2. **Authentication trial.** Validate the recommended maintained auth integration with the actual runtime/database. Record the choice and compatibility result. Do not design password/session cryptography yourself. Implement sign-in/out with the agreed account-provisioning path; public registration is not automatically required.
3. **Tenant-safe data access.** Derive tenant from the verified session. Check it on every delivered read/write. Test signed-out requests and two distinct retailer workspaces. Wrong-role actions must be rejected where such actions exist; do not invent unbuilt admin functionality to claim coverage.
4. **Product and monitor operations.** Build the selected create/view flow with runtime validation, duplicate handling and agreed limits. Coordinate allowed listing rules with Shewon. Schema changes need migrations and the affected owners' review.
5. **Collection handoff.** Agree with Ilmam how authorized enqueue, attempt status and observation storage connect. Keep job enqueue/result persistence consistent under failure. Workers also check monitor ownership/state. Do not fetch the source inside a dashboard read.
6. **Read results.** Return last successful observation separately from latest attempt/error. Extend the shared contract with A/C/D before wiring the real UI. Verify failure preserves previous data.

Handoffs and explain-back

Provide Dilsan example success/error responses early. Give Ilmam migration/setup requirements and ownership rules. Ask Shewon about missing values, variants, sellers and source limitations rather than guessing.

Before review, answer: Where is ownership established? Why can a browser not choose its tenant? What is the difference between a monitor, attempt and observation? How do we preserve a previous price after failure? Which checks did you personally run?

Review, evidence, and AI use

Before each change, explain its purpose and predict how you will check it. Implement a small change; run the check yourself; explain the result. Ask for help after a focused attempt if blocked. Never guess successful results or hide uncertainty.

Use AI as a tutor: "Read my task and the relevant manual section. Ask me to explain it. Help me plan and implement one small step, then interpret my actual check result. Do not complete the whole assignment or change shared contracts without discussion."

For review, record: task key, what changed, why, command or walkthrough performed, actual result, limitations, material AI help, and reviewer. Use a real Jira key in your branch/PR, not the name of this guide. Done means relevant checks pass, a teammate reviews your explanation and change, and the result is integrated and demonstrated. Research tasks finish with reviewed evidence and a verdict, not invented implementation.

Stop and coordinate if an interface changes, a source requires unsupported access, a paid service is needed, or selected scope is no longer credible. Do not switch frameworks, broaden scope, or claim mock data is live. Use current repository source if this PDF is outdated.